



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

命令模式（Command）

一、意图（Intent）

在嵌入式 C 里，命令模式的核心不是“面向对象封装动作”，而是：

把一次操作封成一个可传递、可排队、可延后执行的命令单元，让触发者只负责提交，真正执行者在合适的上下文里落地。

它控制的是：

- 请求以什么数据结构被表示
- 请求由谁触发、由谁真正执行
- 请求在什么时候、在哪个上下文里执行

所以命令模式首先是在控制**执行时机**，其次是在控制**调用者与执行者之间的耦合**。

二、本质 / 动机（Motivation）

嵌入式项目里经常遇到这种场景：

- ISR 想触发一个复杂动作
- 应用层想请求一次外设操作，但不能立刻做
- 多个任务都想访问同一资源，需要串行化
- 某个动作要排队、记录、重试或延迟执行

如果直接写成“谁触发谁就直接调函数”，问题会马上出现。

1. 触发上下文和执行上下文不一致

例如：

- 按键中断只适合上报事件，不适合直接刷屏或写 Flash
- 协议解析线程可以接受命令，但实际硬件访问必须在专门任务里串行执行

这时“谁提出动作”和“谁执行动作”本来就不该是同一个角色。

2. 共享资源需要统一串行化

SPI、I2C、Flash、显示控制器、无线模组这些资源，往往都不适合被多个调用点直接并发操作。

若所有地方都直接调驱动，资源竞争会非常难收敛。

3. 请求需要排队或记录

真实产品里，经常需要：

- 稍后执行
- 放入队列
- 失败后重试
- 记录历史
- 统一做超时和统计

直接函数调用并不擅长表达这些要求。

4. 很多开源 RTOS 和厂家 SDK 默认就这么做

它们并不是让业务层随时随地直接碰底层资源，而是把“要做什么”先变成一个命令或工作项，再交给：

- 服务任务
- 工作队列
- 事件循环
- 后台线程

统一执行。

这正是命令模式在嵌入式里的真实价值。

三、适用性（Applicability）

命令模式特别适合这些条件：

- **需要跨上下文执行**：ISR 触发、线程执行
- **需要延迟执行**：现在提请求，稍后再做
- **需要串行访问共享资源**：一个后台实体统一执行
- **需要排队**：请求先缓存，再按顺序处理

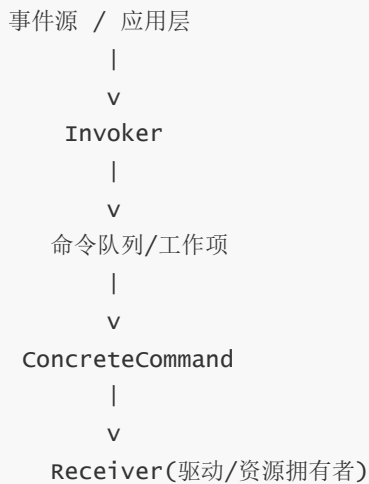
- **需要记录/重放**：测试、日志、恢复、调试
- **需要把触发者与具体实现解耦**：调用方只知道“提交命令”

一句话判断：

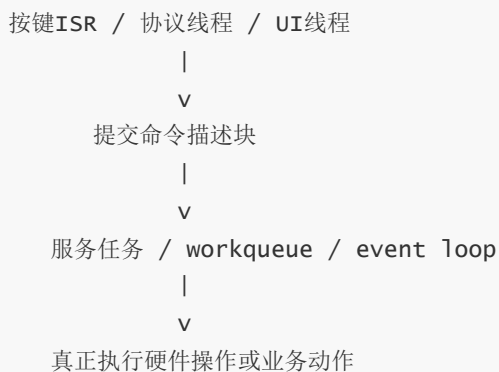
如果系统关心的不只是“做什么”，还关心“何时做、在哪做、按什么顺序做”，命令模式通常就值得考虑。

四、结构（Structure）

从嵌入式 C 的视角，可以把它看成：



更贴近工程实现时，常见关系是：



在这里：

- 提交方不直接碰资源
 - 执行方统一在受控上下文运行
 - 命令本身只是“这次要做什么”的数据载体
-

五、参与者（Participants）

1. Command（命令接口或命令描述）

在嵌入式 C 中，它往往不是一个“类”，而是一种统一的数据表达。

通常会包含：

- 命令 ID
- 参数区
- 目标对象或句柄
- 可选回调
- 状态位
- 超时/时间戳

也就是说，命令首先要把“这次动作”描述清楚。

2. ConcreteCommand（具体命令）

具体命令表示一类明确动作，例如：

- 启动一次采样
- 停止某路 PWM
- 提交一次 Flash 擦写
- 切换某个工作模式

它不一定非要做成很多结构体类型。

在 C 里，更常见的是：

- 一个统一命令结构体 + `cmd_id`
- 配合参数联合体或字段集

3. Receiver（真正执行者）

Receiver 才是真正拥有资源、真正落地动作的地方。

例如：

- Flash 服务任务
- 显示刷新线程
- 总线仲裁层
- 某个具体驱动模块

命令不负责实现细节，它只是把请求组织好并交出去。

4. Invoker（触发器 / 提交器）

Invoker 的职责是：

- 接收外部动作请求
- 保存或转发命令
- 在合适时机触发执行

它不需要知道底层实现细节。

它只需要知道“把什么命令交出去”。

5. Client（发起方）

Client 决定：

- 创建什么命令
- 填什么参数
- 交给哪个 Invoker

在嵌入式里，Client 很可能是：

- 一个 ISR 后半部
- 一个应用任务
- 一个 shell 命令处理函数
- 一个状态机节点

六、协作（Collaborations）

命令模式在嵌入式里的常见执行路径是：

1. 某个上下文产生动作请求
2. Client 组织命令描述块
3. 命令交给 Invoker
4. Invoker 将命令排队、缓存或立即转发
5. Receiver 在自己的上下文中取到命令
6. Receiver 真正执行硬件动作或业务动作
7. 执行完成后可选择上报结果、记录日志或触发下一条命令

这条路径表达的是：

请求的提出与请求的落地被拆开了。

这在嵌入式里非常重要，因为很多系统问题并不是“功能错了”，而是“上下文错了、时机错了、资源归属错了”。

七、效果（Consequences）

收益

1. 调用方与执行方解耦

业务层不需要知道：

- 真正谁执行
- 在哪个线程执行
- 资源什么时候可用
- 执行过程中怎么重试或排队

2. 执行时机可控

命令可以：

- 立刻执行
- 进入队列
- 延时执行
- 由后台线程统一执行

这对 ISR/线程边界、共享资源串行化都很有价值。

3. 天然适合日志、排队和重放

只要命令被表示成数据，就容易：

- 记录
- 转储
- 回放
- 统计
- 测试

4. 更容易做故障隔离

若某类资源必须由专门任务统一访问，那么把所有操作都表示成命令，会比四处直接调驱动安全得多。

代价

1. 引入额外数据结构和调度层

命令池、队列、状态管理都会带来额外 RAM 和复杂度。

2. 简单场景下会显得偏重

如果只是一个局部直接函数调用，没有跨上下文、没有排队、没有资源争用，那么命令模式可能不划算。

3. 需要明确命令所有权

命令由谁申请、谁填写、谁释放、失败后谁回收，这在 C 里必须讲清楚。

八、实现要点 (Implementation)

1. 优先设计“命令数据模型”

在嵌入式里，命令模式成不成立，关键常常不在接口，而在命令结构体是否清楚：

- 命令 ID
- 参数
- 返回路径
- 生命周期
- 所属资源

2. 优先静态分配

命令池、队列、工作项结构更适合静态或预分配方式，避免运行时碎片和不可控失败。

3. 把资源访问权限集中到 Receiver

不要让 Client 一边发命令，一边又偷偷直接碰底层资源。

否则命令模式只剩表面形式，资源仍是失控的。

4. 明确失败语义

队列满、资源忙、执行失败、超时、重复命令等情况都要有统一返回语义。

5. 注意命令上下文切换

如果命令从 ISR 投递到线程执行，接口必须保证：

- ISR 路径无阻塞
- 参数在切换后仍然有效
- 不把临时栈数据错误地交给后台线程

九、典型应用

下面这些都不是课堂虚构例子，而是大量开源系统和厂家 SDK 默认就采用的思路：

1. FreeRTOS 软件定时器服务任务

FreeRTOS 的软件定时器并不是谁启动谁就自己执行回调，而是通过定时器服务任务统一管理，回调也在这个受控上下文里执行。

这非常符合命令模式的思想：

- 外部发起“启动/停止/重载”请求

- 后台统一接收并执行

2. Zephyr 的 workqueue / k_work

很多 Zephyr 场景里，ISR 或普通线程并不直接做重活，而是提交 `work item`，由系统工作队列稍后执行。这就是非常典型的“命令提交”和“后台执行”分离。

3. Linux kernel workqueue / deferred work

Linux 内核大量使用 deferred work，让触发者先提交工作，再由合适上下文执行，避免在不合适的上下文里直接做重操作。

4. 厂家 SDK 里的总线服务任务

很多 MCU 厂家示例工程、模组 SDK、显示/存储中间件里，都会把 SPI、I2C、Flash 这类共享资源的访问变成队列请求，由统一线程串行处理。

这也是命令模式在产品代码里的高频落点。

总结

命令模式在嵌入式 C 里的本质，不是“把函数包一下”，而是把一次动作变成一个可排队、可延后、可跨上下文执行的命令单元。